

一次请求如何经过 SGLang 在线推理框架

OpenAI-compatible serving pipeline (SGLang 0.5.14 / CPU 引擎实测路径)：从 HTTP 请求到流式输出的完整链路

1Client 发起请求 POST /v1/chat/completions (JSON)	▼	OpenAI 兼容协议是"外部契约", 使客户端无需感知内部调度细节。
2API Server 接入 FastAPI 路由 · 鉴权 · 参数校验	▼	渲染 chat 模板; 若 stream=true 则建立 SSE 长连接, 为后续增量下发做准备。
3Tokenizer 文本 → token id 序列	▼	产出 input_length, 是后续调度与显存预算的唯一计量单位。
4Scheduler / Request Queue 入队 · 组批 · continuous batching	▼	按 token 预算与 KV 预算动态组批: 批越大吞吐越高, 但在线场景受延迟约束。
5RadixCache 前缀匹配 基数树上做最长公共前缀查找	▼	★ 逻辑层 / prefix reuse: RadixAttention 在此判定"这段前缀算过没有", 命中即跳到第 6 步的增量计算。
6Prefill (extend) 仅对未命中部分做前向	▼	★命中前缀的 KV 直接复用, prefill 只计算增量 token, 这是 TTFT 下降的主要来源。
7KV Cache 分配 (page) 按固定大小 page/block 分配与回收	▼	★ 物理层: page-sized KV cache 决定 KV 以什么粒度存放与寻址, 前缀复用能否发生取决于存储层是否支持共享。
8Decode 迭代 逐 token 自回归: 读历史 KV → attention → 追加 KV	▼	每次迭代都要读取全部历史 KV, 属 memory-bound; 是 TPOT (每 token 延迟) 的主要构成。
9Sampling temperature / top-p / 约束解码	▼	从 logits 决定下一个 token; 结构化输出约束 (xgrammar) 也作用于此。
10Streaming 输出 增量 SSE 下发	▼	首个增量到达客户端的时刻即 TTFT; 此后每个 token 的间隔即 TPOT。
11Metrics / 可观测性 吞吐 · 缓存命中率 · 队列长度		/metrics 暴露 Prometheus 指标, 本次实验中用其读取前缀缓存命中率。

■ 常规环节 ■ 与本次重点问题直接相关 (RadixAttention / page-sized KV cache / prefix reuse) ■ 面向客户端的观测点

两条关键对应关系

- page-sized KV cache 对应第 7 步: 它是 存储层 的物理单元——把 KV 切成固定大小的 page/block 来分配、共享与回收, 解决的是"KV 怎么放、怎么不碎片化地放"。
- prefix reuse 对应第 5→6 步: 它是 调度/计算层 的逻辑收益——RadixCache 负责"找到可复用的前缀", 其结果直接表现为 prefill 跳过、TTFT 降低。
- 两者是同一件事的两面: 没有可共享的 page 表示, 前缀复用就无法落地; 没有前缀复用, page 机制只是省显存。RadixAttention 是把二者接起来的机制。